Automata Theory

Computational Complexity

CS32022

By Thisara Watawana

13 October 2026

10. Computational Complexity

The Need for Efficiency

- Decidability asks: "Can this problem be solved by an algorithm at all?"
- Complexity theory asks: "Can this problem be solved using realistic amounts of time and memory?"
- A problem may be completely decidable, yet require 2^{1000} steps to solve on an input of length 100.
- An algorithm whose runtime exceeds the lifespan of the universe is computationally useless.
- Complexity shifts our focus from computability (solvable vs. unsolvable) to tractability (feasible vs. infeasible).

10. Computational Complexity

Measuring Time on Turing Machines

- Let M be a deterministic Turing Machine that halts on all inputs.
- The running time or time complexity of M is the function f: $\mathbb{N} \to \mathbb{N}$, where f(n) is the maximum number of steps that M uses on any input of length n (worst-case analysis).
- **Definition of Time Complexity Classes:**

  $$\text{TIME}(t(n)) = \{ L \mid L \text{ is a language decided by an } O(t(n))\text{-time deterministic TM} \}$$

- This formalizes the standard Big-O runtime classifications ($O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$) on a rigorous mathematical model.

10. Computational Complexity

The Class P (Polynomial Time)

- **Definition:** $P = \bigcup_{k \geq 1} TIME(n^k)$
- P is the class of languages that are decidable in polynomial time by a Deterministic single-tape Turing Machine.
- **The Cobham-Edmonds Thesis:** Polynomial time ($O(n^k)$) mathematically formalizes the boundary of "tractable" or practically feasible computation.
- Problems decidable in polynomial time scale with improvements in computing hardware; exponential problems do not.

10. Computational Complexity

Classic DSA Problems in P

- **Shortest Path:** Given a directed graph, edge weights, and two nodes s and t, find the shortest path. Solved in polynomial time by Dijkstra's algorithm ($O(|E| + |V| \log |V|)$) or BFS ($O(|V| + |E|)$).
- **Minimum Spanning Tree:** Solved by Prim's or Kruskal's algorithm in $O(|E| \log |V|)$.
- **Sorting & Searching:** MergeSort runs in $O(n \log n)$ steps; Binary Search runs in $O(\log n)$.
- **Graph Connectivity:** Determining whether a path exists between s and t (PATH problem) is decidable in $O(n^2)$ time.

10. Computational Complexity

The Class NP: Definition 1 (Nondeterministic TM)

- **Critical Rule:** NP does NOT stand for "Non-Polynomial." NP stands for Nondeterministic Polynomial Time.

- **Definition:** NP = $\bigcup_{k \geq 1}$ NTIME(n^k)

  NP is the class of languages decidable in polynomial time by a Nondeterministic Turing Machine (NTM).

- **Recall NTM Execution:** An NTM branches at every step. It accepts an input if at least one computation path reaches q_{accept} within n^k steps.

10. Computational Complexity

The Class NP: Definition 2 (Polynomial Verifier)

- **An equivalent, more intuitive definition of NP:** A language L is in NP if and only if it has a polynomial-time deterministic verifier.
- **A verifier *V*** is an algorithm that takes two inputs:
  - The input string w (the problem instance).
  - A certificate or witness c (the proposed solution/proof).
- **Formal Criterion:**
  $w \in L \Leftrightarrow$ there exists a polynomial-length certificate c such that $V(\langle w, c \rangle)$ accepts in polynomial time
- **Summary:** NP is the set of all decision problems whose solutions can be verified in polynomial time.

10. Computational Complexity

The P versus NP Question

- Because every deterministic TM is trivially a nondeterministic TM with only one branch: **$P \subseteq NP$**
- **The Million-Dollar Question (Clay Millennium Prize):** Does P = NP?
  Informally: If a problem's solution can be verified efficiently, can it always be found efficiently?
- **Consensus:** Most theoretical computer scientists believe P ≠ NP, but no formal proof currently exists.
- If P = NP, then every puzzle that is easy to check is equally easy to solve, breaking modern public-key cryptography.

10. Computational Complexity

Polynomial-Time Reductions (A ≤□ B)

- **Connecting Reducibility to Complexity:** In computability theory, we used mapping reductions (A ≤□ B) to transfer undecidability. In complexity theory, we add an execution budget: the reduction function must compute its mapping in polynomial time.
- **Definition:** Language A is polynomial-time mapping reducible to language B (A ≤□ B) if a polynomial-time computable function f: $\Sigma^* \rightarrow \Sigma^*$ exists such that:
  $w \in A \Leftrightarrow f(w) \in B$
- **Key Theorem:** If A ≤□ B and B ∈ P, then A ∈ P.

10. Computational Complexity

NP-Hard and NP-Complete

- **NP-Hard:** A language *H* is NP-hard if every problem in NP reduces to it in polynomial time:
  $\forall A \in \text{NP}, A \leq_{\square} H$
  (NP-hard problems are at least as hard as the hardest problems in NP; they do not have to be inside NP themselves).
- **NP-Complete (NPC):** A language *B* is NP-Complete if:
  1. $B \in$ NP (solutions can be verified in polynomial time), AND
  2. *B* is NP-hard (every problem in NP reduces to *B* in polynomial time).
- **Significance:** The NP-complete problems are the hardest problems in NP. If any single NP-complete problem can be solved in polynomial time, then P = NP.

Questions?

thisaraw@boffin.lk